Fundação Escola de Comércio Álvares Penteado - FECAP
CCOMP - 6º SEMESTRE

Kauan Rocha Dias - 24026492
Bruno da Silva Ribeiro – 24025958
Guilherme Melo da Costa - 24025906
Isadora Teixeira Santoma - 24026458

Entrega 2 - Sistemas embarcados e robótica

Relatório técnico de prova de conceito (PoC)

Tecnologia que detecta faltas e rastreamento de pontos

Introdução e Descrição do dispositivo

O projeto consiste no desenvolvimento e na validação da Prova de Conceito de um dispositivo eletrônico embarcado, acoplado à rede da quadra de tênis, voltado à detecção automática de impactos e, em especial, da falta conhecida como *let*. O *let* ocorre quando a bola de tênis bate na rede e, ainda assim, passa para o outro lado da quadra, situação que, em treinos e partidas sem árbitro, costuma gerar dúvidas e erros de arbitragem.

Para a verificar a falta, o sistema emprega discos piezoelétricos, que geram uma pequena tensão proporcional ao impacto mecânico sofrido. O sinal é lido pelas entradas analógicas de um Arduino R4 WiFi. Os eventos detectados alimentam um placar web que bloqueia momentaneamente a pontuação quando um *let* é identificado.

Tecnologias e ferramentas utilizadas

Hardware:

- Arduino UNO R4 wifi
- 3 Piezos elétricos
- Protoboard

IDE:

- Arduino IDE (C++)

Front-end:

- React

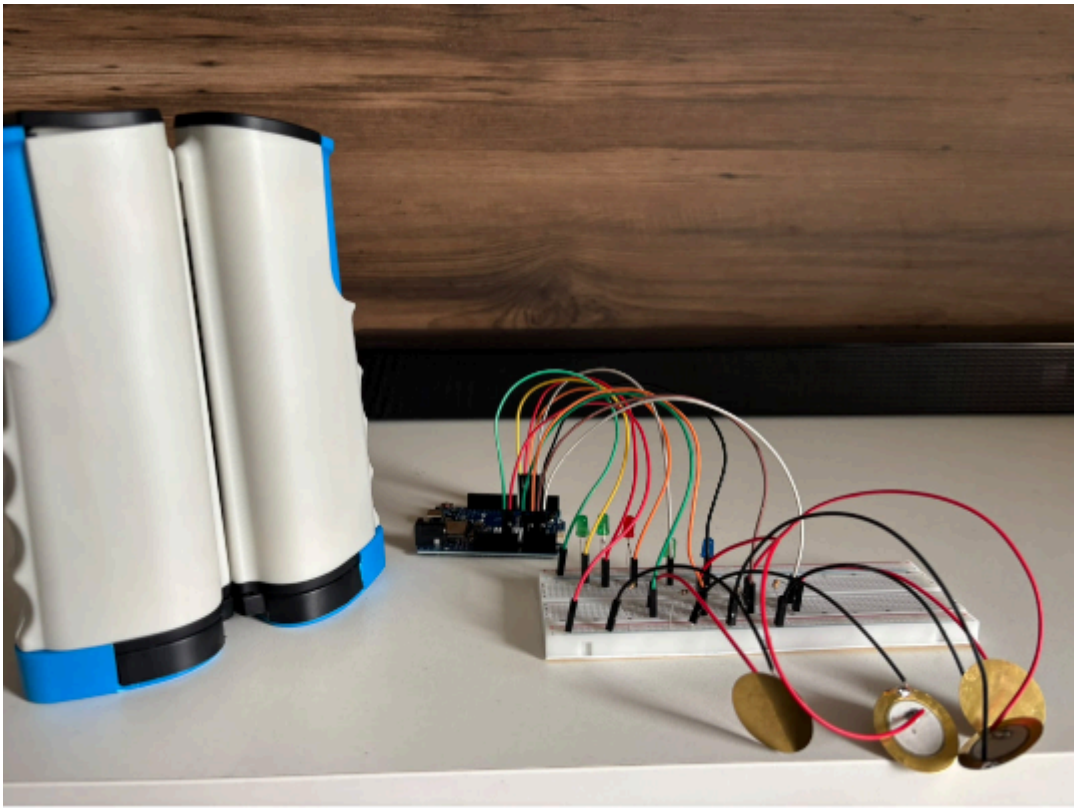
Back-end:

FastAPI

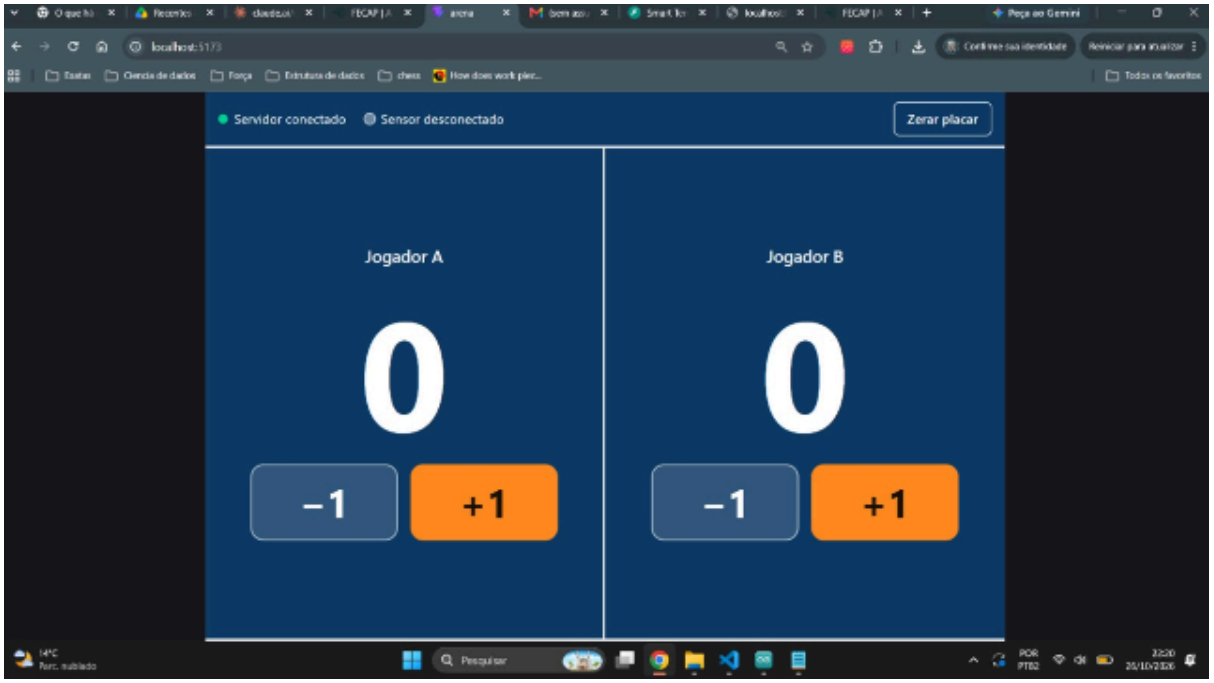
Demonstração Prática do protótipo

Para demonstrar o funcionamento, disponibilizaram-se os seguintes registros:

- Foto da montagem: circuito (Arduino Uno, protoboard, LEDs e piezos) ao lado da rede de testes do *let*;



- Capturas de tela da aplicação: placar com Jogador A e Jogador B e indicador de status do servidor, e tela vermelha escrito “LET!” com a mensagem de placar bloqueado;





- Vídeo de Demonstração: [https://youtu.be/3M6TrdwRCeI]

Códigos do sistema

Codigo do Arduino:

```
/*
  Smart Tennis Arena - Sensor de LET via USB (Arduino UNO R4 WiFi + 2 piezos)

  Piezos em A1 e A2, LED único no pino 3.
  A cada "let" é enviada uma linha JSON pela serial:

{"event":"let","timestamp":123456,"sensor":2,"intensity":150,"peaks":[150,40]}

  O servidor só trata como evento as linhas que começam com "{". O log periódico
  e as mensagens de início (começam com "#") são ignorados.
  Feche o Serial Monitor da Arduino IDE antes de rodar o servidor.
*/

// ===== PIEZOS =====
const uint8_t NUM_SENSORS = 2;
const int     PIEZO_PINS[NUM_SENSORS] = {A1, A2};
// ID enviado ao servidor para cada piezo (mantém a numeração antiga: A1 = 2, A2 = 3)
const uint8_t SENSOR_ID[NUM_SENSORS]  = {2, 3};

// ===== LED =====
const int      LED_PIN     = 3;
const uint32_t LED_HOLD_MS = 200;   // tempo que o LED fica aceso após um let

// ===== DETECÇÃO DE LET =====
const int      THRESHOLD      = 90;   // 0..1023; só dispara acima disso (ruído ~75, batidas ~110+)
const int      REARM_LEVEL    = 60;   // só rearma quando todos os sensores caírem abaixo disso
```

```
const uint16_t PEAK_WINDOW_MS = 8;    // janela para captar o pico
real
const uint32_t COOLDOWN_MS    = 500;  // tempo mínimo entre dois
"let"

// ===== LOG =====
const bool DEBUG_MODE    = false;     // true = imprime leituras rápidas
para calibrar
const bool LOG_PERIODICO = true;      // log a cada 5 s (ignorado pelo
servidor)
const unsigned long INTERVALO_LOG = 5000;

unsigned long ultimoLog = 0;
unsigned long lastLetAt = 0;
unsigned long ledStart  = 0;
bool ledOn = false;
bool armed = true;

void sendLet(uint8_t sensorIndex, int intensity, const int* peaks) {
  char json[128];
  int n = snprintf(json, sizeof(json),

"{\"event\":\"let\",\"timestamp\":%lu,\"sensor\":%u,\"intensity\":%d,\"peaks\
":[",
                   millis(), (unsigned)SENSOR_ID[sensorIndex], intensity);
  for (uint8_t i = 0; i < NUM_SENSORS; i++) {
    n += snprintf(json + n, sizeof(json) - n, i ? ",%d" : "%d", peaks[i]);
  }
  snprintf(json + n, sizeof(json) - n, "]}");
  Serial.println(json);
}

void setup() {
  Serial.begin(115200);  // igual ao padrão do servidor (SERIAL_BAUD)

#ifndef ARDUINO_ARCH_AVR
```

```
  analogReadResolution(10);  // o UNO R4 já usa 10 bits por padrão; a linha só deixa isso explícito
#endif

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.println("# Smart Tennis Arena pronto (UNO R4 WiFi, A1/A2)");
}

void loop() {
  int values[NUM_SENSORS];
  bool triggered = false;
  bool quiet = true;

  for (uint8_t i = 0; i < NUM_SENSORS; i++) {
    values[i] = analogRead(PIEZO_PINS[i]);
    if (values[i] > THRESHOLD)   triggered = true;
    if (values[i] > REARM_LEVEL) quiet = false;
  }

  // Apaga o LED depois do tempo de hold
  if (ledOn && millis() - ledStart >= LED_HOLD_MS) {
    ledOn = false;
    digitalWrite(LED_PIN, LOW);
  }

  if (DEBUG_MODE) {
    Serial.print(values[0]); Serial.print('\t');
    Serial.println(values[1]);
    delay(20);
    return;
  }

  if (LOG_PERIODICO && millis() - ultimoLog >= INTERVALO_LOG) {
    ultimoLog = millis();
    Serial.print("# piezos A1/A2: ");
```

```
    Serial.print(values[0]); Serial.print(" / ");
    Serial.println(values[1]);
  }

  // Rearma só quando o sinal voltou ao repouso e o cooldown passou
  if (!armed && quiet && millis() - lastLetAt >= COOLDOWN_MS) armed
= true;

  if (!triggered || !armed) return;

  // Registra o pico de cada sensor para saber qual recebeu a batida
mais forte
  int peaks[NUM_SENSORS];
  for (uint8_t i = 0; i < NUM_SENSORS; i++) peaks[i] = values[i];

  unsigned long start = millis();
  while (millis() - start < PEAK_WINDOW_MS) {
    for (uint8_t i = 0; i < NUM_SENSORS; i++) {
      int v = analogRead(PIEZO_PINS[i]);
      if (v > peaks[i]) peaks[i] = v;
    }
  }

  uint8_t strongest = 0;
  for (uint8_t i = 1; i < NUM_SENSORS; i++) {
    if (peaks[i] > peaks[strongest]) strongest = i;
  }

  lastLetAt = millis();
  armed = false;

  digitalWrite(LED_PIN, HIGH);
  ledOn = true;
  ledStart = millis();

  sendLet(strongest, peaks[strongest], peaks);
}
```